Unit 5

Topics

- The bias variance tradeoff: Error due to bias, Error due to variance,
- Two extreme cases of bias/variance tradeoff: Underfitting, Overfitting,
- How bias/variance play into error functions,
- K folds cross-validation,
- Grid searching,
- Visualizing training error versus cross-validation error

The Bias Variance Tradeoff

Error due to bias

- When speaking of errors due to Bias, we are speaking of the difference between the expected prediction of our model and the actual (correct) value, which we are trying to predict.
- Bias, in effect, measures how far, in general, our model's predictions are from the correct value.
- Think about bias as simply being the difference between a predicted value and the actual value.
- For example, consider that our model, represented as F(x), predicts the value of 29 as follows:
- F(29) = 88
- Here, the value of 29 should have been predicted at 79, then:
- Bias(29) = 88-79 = 9
- If a machine learning model tends to be very accurate in its prediction (regression or classification), then it is considered a low Bias model, whereas if the model is more often than not wrong, it is considered to be a high bias model.
- Bias is a measure to judge models on the basis of accuracy or just how correct the model is on an average.

Error due to variance

- An error due to variance is dependent on the variability of a model prediction for a given data point.
- Imagine that you repeat the machine learning model building process over and over. The variance is measured by looking at how much the predictions for a fixed point vary between different end results.
- To imagine variance in your head, think about a population of data points. If you were to take randomized samples over and over, how drastically would your machine learning model change or fit differently each time.
- If the model does not change much between samples, the model would be considered a low variance model. If your model changes drastically between samples, then that model would be considered a high variance model.
- Variance is a great measure to judge our model on the basis of generalizability.
- If our model has a low variance, we can expect it to behave in a certain way when set into the wild and predict values without human supervision.
- Our goal is to optimize both bias and variance. Ideally, we are looking for the lowest possible variance and bias.

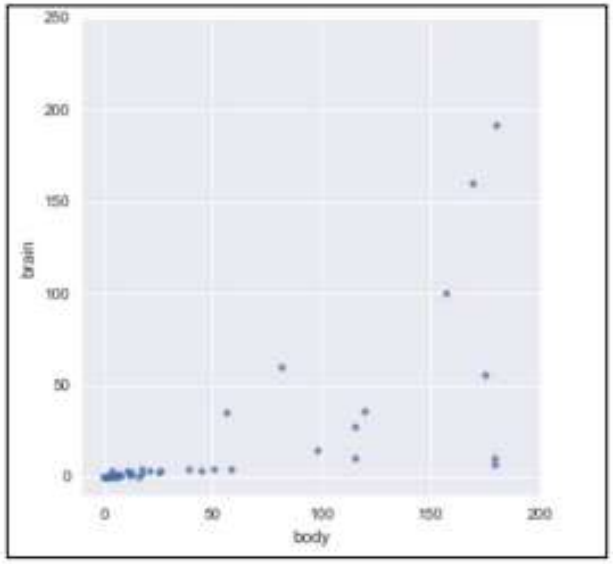
Example – comparing body and brain weight of mammals

- Imagine that we are considering a relationship between the brain weight of mammals and their corresponding body weights.
- A hypothesis might read that there is a positive correlation between the two (as one goes up, so does the other).
- But how strong is this relationship? Is it even linear? Perhaps, as the brain weight increases, there is a logarithmic or quadratic increase in body weight.
- # # Exploring the Bias-Variance Tradeoff
- import pandas as pd
- import numpy as np
- import seaborn as sns
- %matplotlib inline
- I will be using a module, called seaborn, to visualize data points as a scatter plot and also to graph linear (and higher polynomial) regression models:
- # Brain and body weight
- # This is a [dataset]) of the average weight of the body and the brain for 62 mammal species. Let's read it into pandas and take a quick look:

```
df.head()
```

	brain	body
id		
1	3.385	44.5
2	0.480	15.5
3	1.350	8.1
4	465.000	423.0
5	36.330	119.5

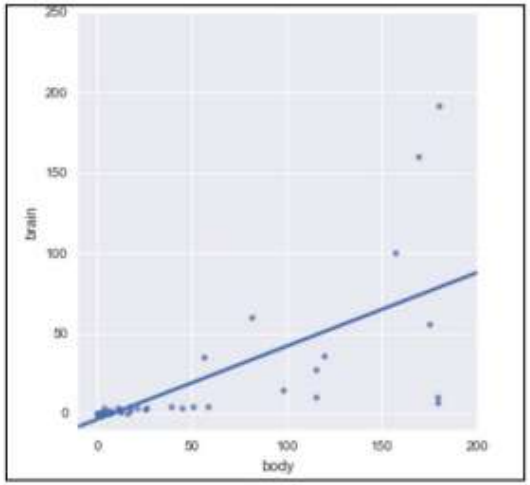
- We are going to take a small subset of the samples to exacerbate the visual representations of bias and variance, as follows:
- # We're going to focus on a smaller subset in which the body weight is less than 200:
- df = df[df.body < 200]
- df.shape (51, 2)
- We're actually going to pretend that there are only 51 mammal species in existence. In other words, we are pretending that this is the entire dataset of brain and body weights for every known mammal species.
- # Let's create a scatterplot
- sns.lmplot(x='body', y='brain', data=df, ci=None, fit_reg=False)
- sns.plt.xlim(-10, 200)
- sns.plt.ylim(-10, 250)

- 

Scatter plot of mammalian brain and body weights

There appears to be a relationship between brain and body weight for mammals.
So far, we might assume that it is a positive correlation.
Now, let's throw in a linear regression into the mix.
Let's use seaborn to make and plot a first degree polynomial (linear) regression.

```
sns.lmplot(x='body', y='brain', data=df, ci=None)
sns.plt.xlim(-10, 200)
sns.plt.ylim(-10, 250)
```



Same scatter plot as before with a linear regression visualization put in

- Now, let's pretend that a new mammal species is discovered.
- We measure the body weight of every member of this species that we can find, and calculate an average body weight of 100.
- We want to predict the average brain weight of this species (rather than measuring it directly). Using this line, we might predict a brain weight of about 45.

- Something you might note is that this line isn't that close to the data points in the graph, so, maybe it isn't the best model to use!
- You might argue that the bias is too high.
- And I would agree! Linear regression models tend to have high bias, but linear regression also has something up its sleeve—it has a very low variance.
- However, what does that really mean?
- Let's say that we take our entire population of mammals and randomly split them into two samples, as follows:

- # randomly assign every row to either sample 1 or sample 2 df['sample'] = np.random.randint(1, 3, len(df))
- df.head()
- Include a new sample column:

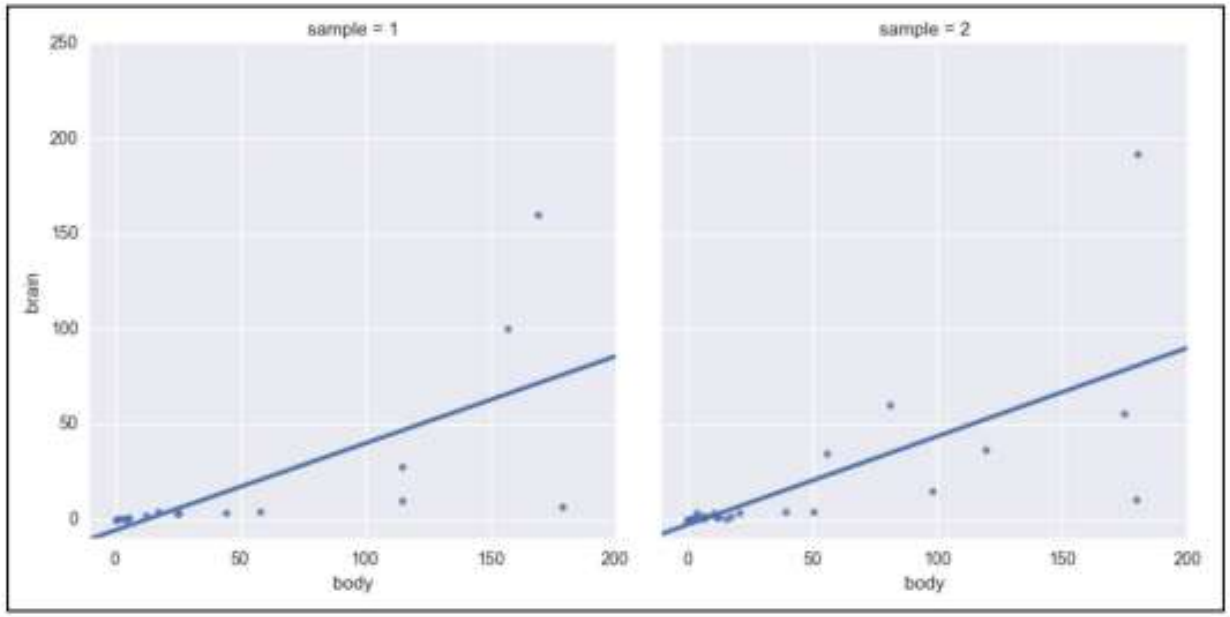
	brain	body	sample
id			
1	3.385	44.5	1
2	0.480	15.5	2
3	1.350	8.1	2
5	36.330	119.5	2
6	27.660	115.0	1

- # Compare the two samples,
- they are fairly different!
- df.groupby('sample')
  [['brain', 'body']].mean()

	brain	body
sample		
1	18.113778	52.068889
2	13.323364	34.669091

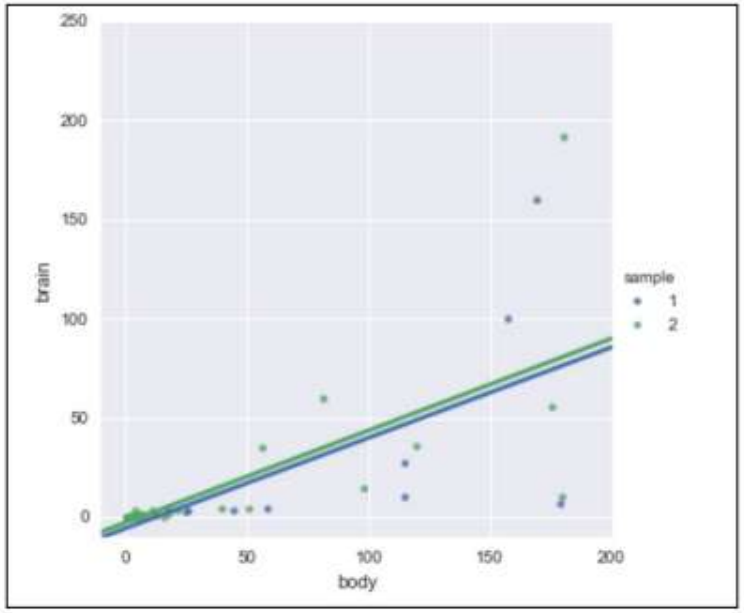
We can now tell seaborn to create two plots, in which the left plot only uses the data from sample 1 and the right plot only uses the data from sample 2:

```
# col='sample' subsets the data by sample and creates two
# separate plots
sns.lmplot(x='body', y='brain', data=df, ci=None, col='sample')
sns.plt.xlim(-10, 200)
sns.plt.ylim(-10, 250)
```

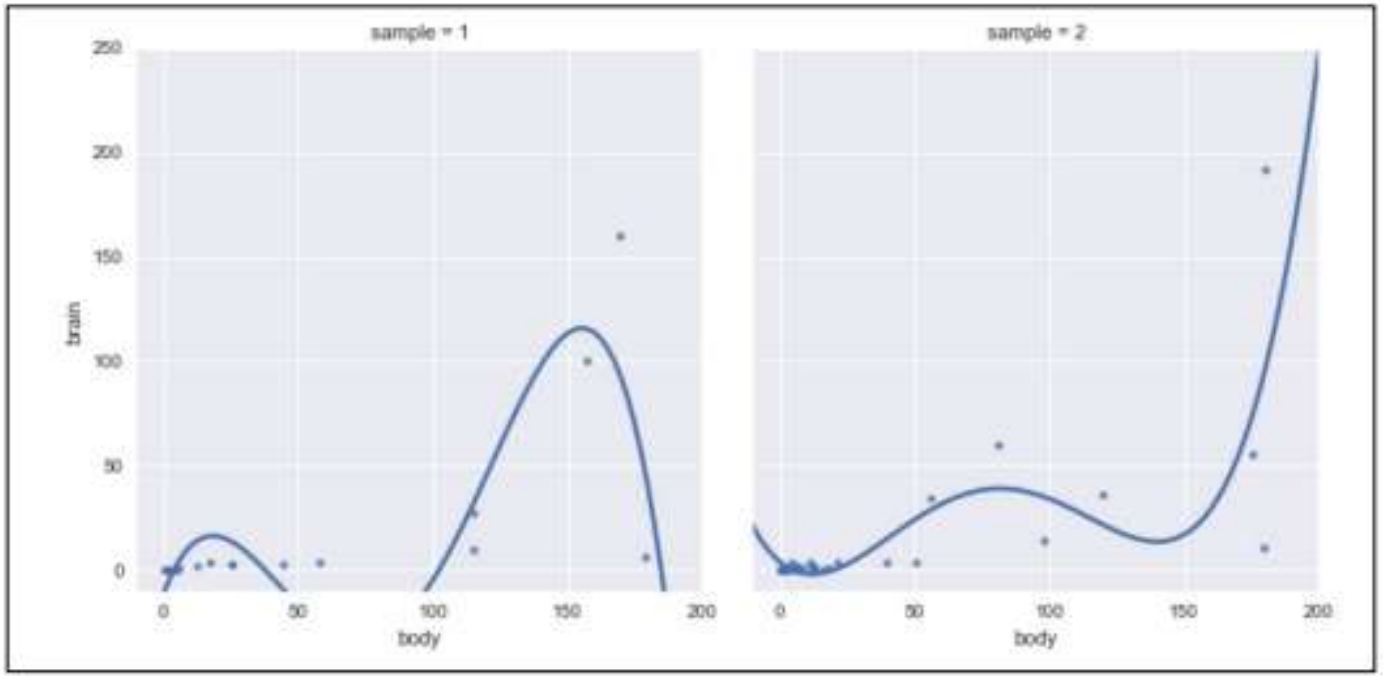


Side-by-side scatter plots of linear regressions for samples 1 and 2

- They barely look different, right? If you look closely, you will note that not a single data point is shared between the samples and yet the line looks almost identical.
- To further show this point, let's put both the lines of best fit in the same graph and use colors to separate the samples, as illustrated:
- # hue='sample' subsets the data by sample and creates a
- # single plot
- sns.lmplot(x='body', y='brain',
  data=df, ci=None, hue='sample')
  sns.plt.xlim(-10, 200)
  sns.plt.ylim(-10, 250)



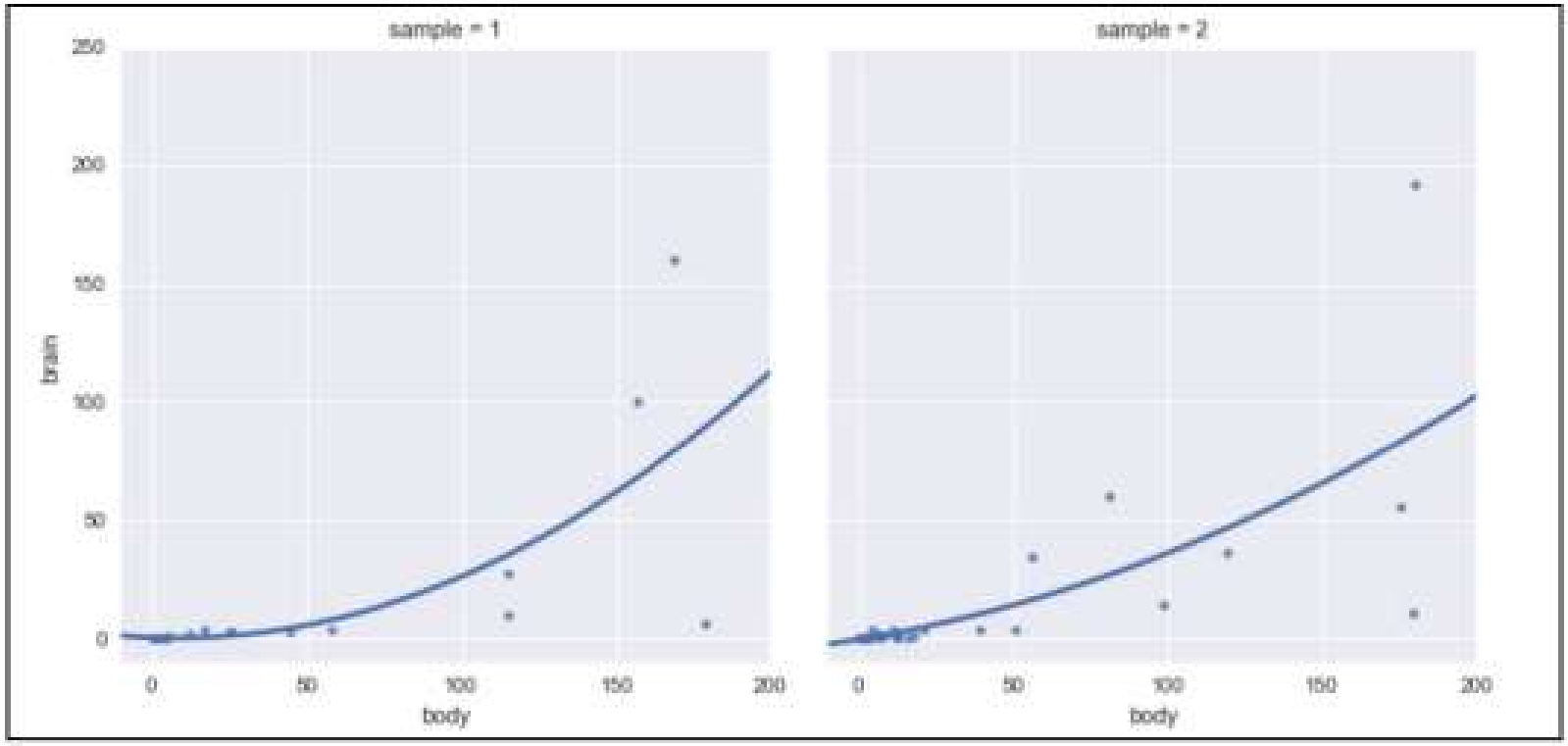
- The line looks pretty similar between the two plots, despite the fact that they used separate samples of data.
- In both the cases, we would predict a brain weight of about 45.
- The fact that even though the linear regression was given to completely distinct datasets pulled from the same population, it produced a very similar line, suggesting that the model is of low variance.
- What if we increased our model's complexity and allowed it to learn more? Instead of fitting a line, let's let seaborn fit a fourth degree polynomial (a quartic polynomial).
- By adding to the degree of the polynomial, the graph will be able to make twists and turns in order to fit our data better, as shown:
- # What would a low bias, high variance model look like? Let's try polynomial regression, with an fourth order polynomial:
- sns.lmplot(x='body', y='brain', data=df, ci=None, \ col='sample', order=4)
- sns.plt.xlim(-10, 200)
- sns.plt.ylim(-10, 250)



Using a quartic polynomial for regression purposes

- Note how, for two distinct samples from the same population, the quartic polynomial looks vastly different.
- This is a sign of high variance.
- This model is low bias because it matches our data well! However, it's high variance because the models are widely different depending upon which points happen to be in the sample.
- (For a body weight of 100, the brain weight prediction would either be 40 or 0, depending upon which data happened to be in the sample.)

- Our polynomial is also unaware of the general relationship of the data. It seems obvious that there is a positive correlation between the brain and body weight of mammals.
- However, in our quartic polynomials, this relationship is nowhere to be found and is unreliable. In our first sample (the graph on the left), the polynomial ends up shooting downwards, while in the second graph, the graph is going upwards towards the end.
- Our model is unpredictable and can behave wildly different depending on the given training set. It is our job, as data scientists, to find a middle ground.
- Perhaps we can create a model that has less bias than the linear model, and less variance than the fourth order polynomial?
- # Let's try a second order polynomial instead:
- sns.lmplot(x='body', y='brain', data=df, ci=None, col='sample', order=2)
- sns.plt.xlim(-10, 200)
- sns.plt.ylim(-10, 250)



Scatter plot using a quadratic polynomial as our estimator

This plot seems to have a good balance of bias and variance.

Two extreme cases of Bias/Variance Tradeoff

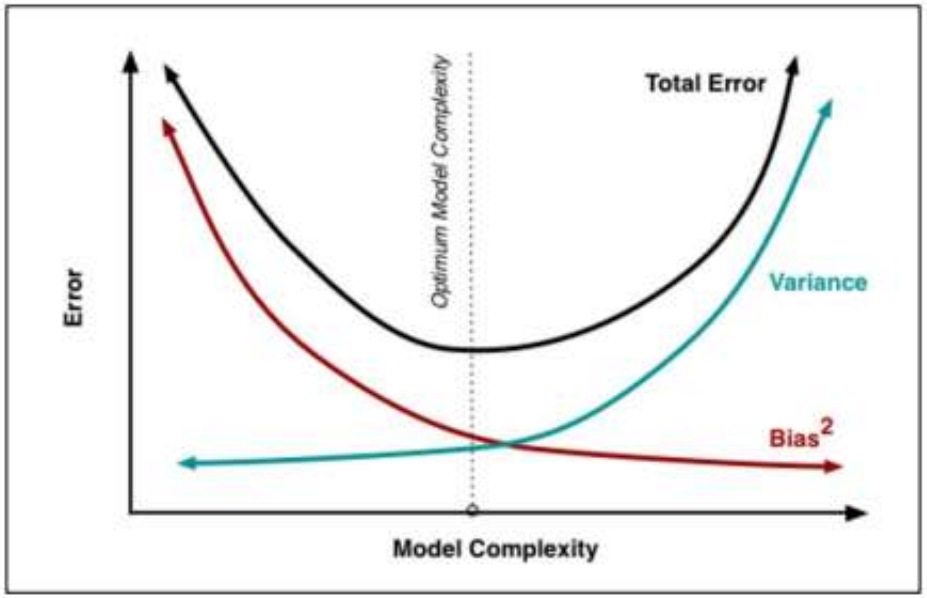
- **Underfitting:**
- Underfitting occurs when our models make little to no attempt to fit our data.
- Models that are high bias and low variance are prone to underfitting. In the case of the mammal brain/body weight example, the linear regression is underfitting our data.
- While we have a general shape of the relationship, we are left with a high bias.
- If your learning algorithm shows high bias and/or is underfitting, the following suggestions may help:
- Use more features: Try including new features into the model if it helps with our predictive power.
- Try a more complicated model: Adding complexity to your model can help improve bias. An overly complicated model will hurt too!

- **Overfitting:**
- Overfitting is the result of the model trying too hard to fit into the training set, resulting in a lower bias but a much higher variance.
- Models that are low bias and high variance are prone to overfitting.
- In the case of the mammal brain/body weight example, the fourth degree polynomial (quartic) regression is overfitting our data.
- If your learning algorithm shows high variance and/or is overfitting, the following suggestions may help:
- Use fewer features: Using fewer features can decrease our variance and prevent overfitting
- Fit on more training samples: Using more training data points in our cross-validation can reduce the effect of overfitting, and improve our high variance estimator

How Bias/Variance play into error functions

- Error functions (that measure how incorrect our models are) can be thought of as functions of bias, variance, and irreducible error. Mathematically put, the error of predicting a dataset using our supervised learning model might look as follows:
- $Error\ (x) = Bias^2 + Variance + Irreducible\ Error$
- Here, $Bias^2$ is our bias term squared (arises when simplifying the mentioned statement from more complicated equations), Variance is a measurement of how much our model fitting varies between randomized samples.

- Simply put, both bias and variance contribute to errors.
- As we increase our model complexity (for example, go from a linear regression to an eighth degree polynomial regression or grow our decision trees deeper), we find that Bias2 decreases, variance increases, and the total error of the model forms a parabolic shape, as illustrated:



- Our goal, as data scientists, is to find the sweet spot that optimizes our model complexity.
- It is easy to overfit our data.
- To combat overfitting in practice, we should always use cross-validation (splitting up datasets iteratively and retraining models and averaging metrics) to get the best predictor of an error.
- To illustrate this point, I will introduce (quickly) a new supervised algorithm and demonstrate the bias/variance tradeoff visually.
- We will be using the K-Nearest Neighbors (KNN) algorithm, which is a supervised learning algorithm that uses a lookalike paradigm, which means that it makes predictions based on similar data points seen in the past.

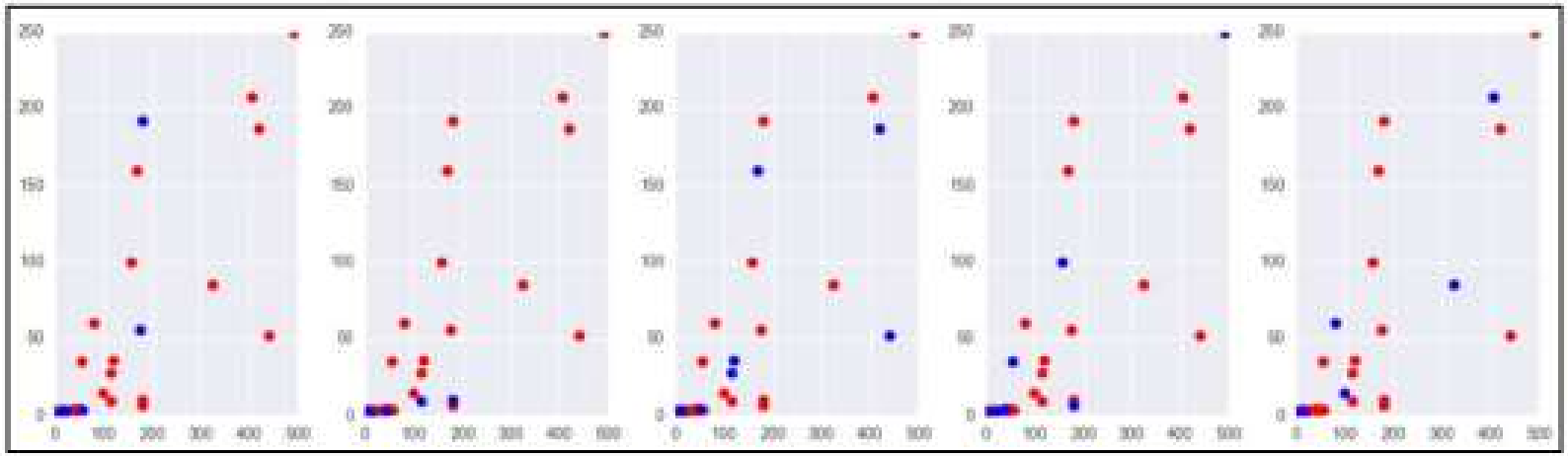
- KNN has a complexity input, K, which represents how many similar data points to compare to.
- If K = 3, then, for a given input, we look to the nearest three data points and use them for our prediction.
- In this case, K represents our model complexity.
- from sklearn.neighbors import KNeighborsClassifier
- # read in the iris data
- from sklearn.datasets import load_iris
- iris = load_iris()
- X, y = iris.data, iris.target
- So, we have our X and our y. A great way to overfit a model is to train and predict on the exact same data.
- knn = KNeighborsClassifier(n_neighbors=1)
- knn.fit(X, y)
- knn.score(X, y)
- 1.0
- Wow, a 100% accuracy?!
- This is too good to be true.
- By training and predicting on the same data, we are essentially telling our data to purely memorize the training set and spit it back to us (this is called our training error).

K folds cross-validation

- K folds cross-validation is a much better estimator of our model's performance, even more so than our train-test split. Here's how it works:
- 1. We will take a finite number of equal slices of our data (usually 3, 5, or 10). Assume that this number is called k.
- 2. For each "fold" of the cross-validation, we will treat k-1 of the sections as the training set, and the remaining section as our test set.
- 3. For the remaining folds, a different arrangement of k-1 sections is considered for our training set and a different section is our training set.
- 4. We compute a set metric for each fold of the cross-validation.
- 5. We average our scores at the end.

- Cross-validation is effectively using multiple train-test splits being done on the same dataset.
- This is done for a few reasons, but mainly because cross-validation is the most honest estimate of our model's out of the sample error.
- To explain this visually, let's look at our mammal brain and body weight example for a second.
- The following code manually creates a five-fold cross-validation, wherein five different training and test sets are made from the same population:
- from sklearn.cross_validation import KFold
- df = pd.read_table('http://people.sc.fsu.edu/~jburkardt/datasets/regression/x01.txt', sep='\s+', skiprows=33, names=['id','brain','body'])
- df = df[df.brain < 300][df.body < 500]

- # limit points for visibility
- nfolds = 5
- fig, axes = plt.subplots(1, nfolds, figsize=(14,4))
- for i, fold in enumerate(KFold(len(df), n_folds=nfolds, shuffle=True)): training, validation = fold
  - x, y = df.iloc[training]['body'], df.iloc[training]['brain']
  - axes[i].plot(x, y, 'ro')
  - x, y = df.iloc[validation]['body'], df.iloc[validation]['brain'] axes[i].plot(x, y, 'bo')
- plt.tight_layout()



Five-fold cross-validation: red = training sets, blue = test sets

- Here, each graph shows the exact same population of mammals, but the dots are colored red if they belong to the training set of that fold and blue if they belong to the testing set.
- By doing this, we are obtaining five different instances of the same machine learning model in order to see if performance remains consistent across the folds.
- If you stare at the dots long enough, you will note that each dot appears in a training set exactly four times (k – 1), while the same dot appears in a test set exactly once and only once.

- Some features of K-fold cross-validation include the following:
- • It is a more accurate estimate of the OOS prediction error than a single train-test split because it is taking several independent train-test splits and averaging the results together.
- • It is a more efficient use of data than single train-test splits because the entire dataset is being used for multiple train-test splits instead of just one.
- • Each record in our dataset is used for both training and testing.
- • This method presents a clear tradeoff between efficiency and computational expense. A 10-fold CV is 10x more expensive computationally than a single train/test split.
- • This method can be used for parameter tuning and model selection.

- Basically, whenever we wish to test a model on a set of data, whether we just completed tuning some parameters or feature engineering, a k-fold cross-validation is an excellent way to estimate the performance on our model.
- Of course, sklearn comes with an easier-to-use cross-validation module, called cross_val_score, which automatically splits up our dataset for us, runs the model on each fold, and gives us a neat and tidy output of results:

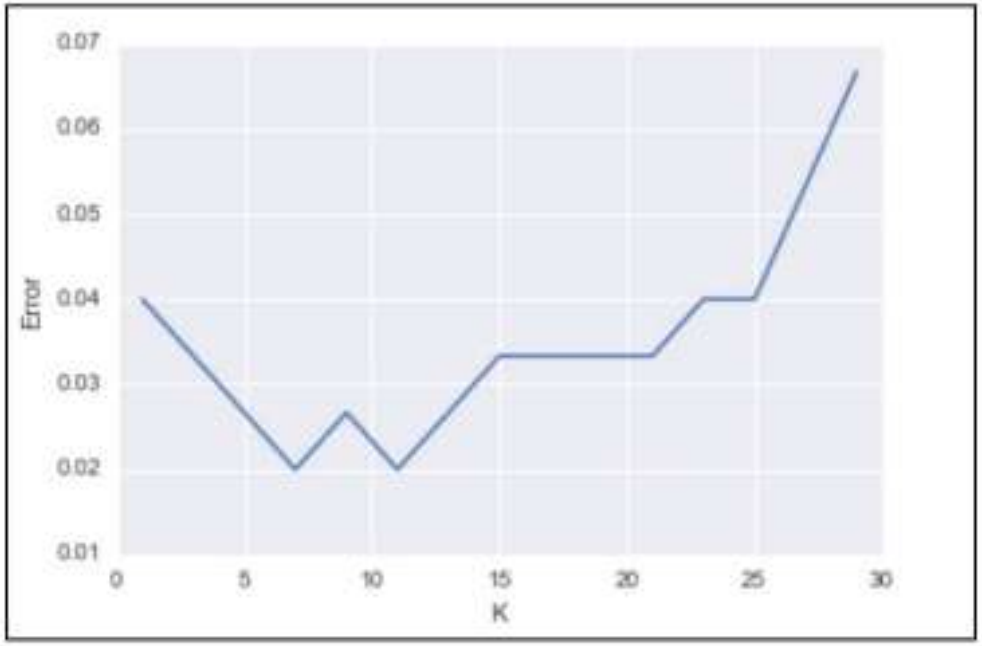
- # Using a training set and test set is so important
- # Just as important is cross validation. Remember cross validation
- # is using several different train test splits and
- # averaging your results!

- ## CROSS-VALIDATION

- # check CV score for K=1
- from sklearn.cross_validation import cross_val_score, train_test_split tree = KNeighborsClassifier(n_neighbors=1)
- scores = cross_val_score(tree, X, y, cv=5, scoring='accuracy')
- scores.mean()
- 0.95999999999
- Which is a much more reasonable accuracy than our previous score of 1.
- Remember that we are not getting 100% accuracy anymore, because we have a distinct training and test set.
- The data points that KNN has never seen the test points and therefore cannot match them exactly to themselves.

- Let's try cross-validating KNN with K=5 (increasing our model's complexity), as shown:
- # check CV score for K=5
- knn = KNeighborsClassifier(n_neighbors=5)
- scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
- scores
- np.mean(scores)
- 0.97333333

- Even better! So, now we have to find the best K?
- The best K is the one that maximizes our accuracy.
- Let's try a few:
- # search for an optimal value of K
- k_range = range(1, 30, 2) # [1, 3, 5, 7, …, 27, 29] errors = []
  - for k in k_range: knn = KNeighborsClassifier(n_neighbors=k)
  - # instantiate a KNN with k neighbors
  - scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
  - # get our five accuracy scores
  - accuracy = np.mean(scores)
  - # average them together
  - error = 1 – accuracy
  - # get our error, which is 1 minus the accuracy
  - errors.append(error)
  - # keep track of a list of errors
- We now have an error value (1 – accuracy) for each value of K (1, 3, 5, 7, 9.., .., 29):
- # plot the K values (x-axis) versus the 5-fold CV score (y-axis)
  - plt.figure()
  - plt.plot(k_range, errors)
  - plt.xlabel('K') plt.ylabel('Error')



Graph of errors of KNN model against KNN's complexity, represented by the value of K

- Compare this graph to the previous graph of model complexity and bias/variance.
  Toward the left, our graph has a higher bias and is underfitting.
  As we increased our model's complexity, the error term began to go down, but after a while, our model became overly complex, and the high variance kicked in, making our error term go back up.
  It seems that the optimal value of K is between 6 and 10.

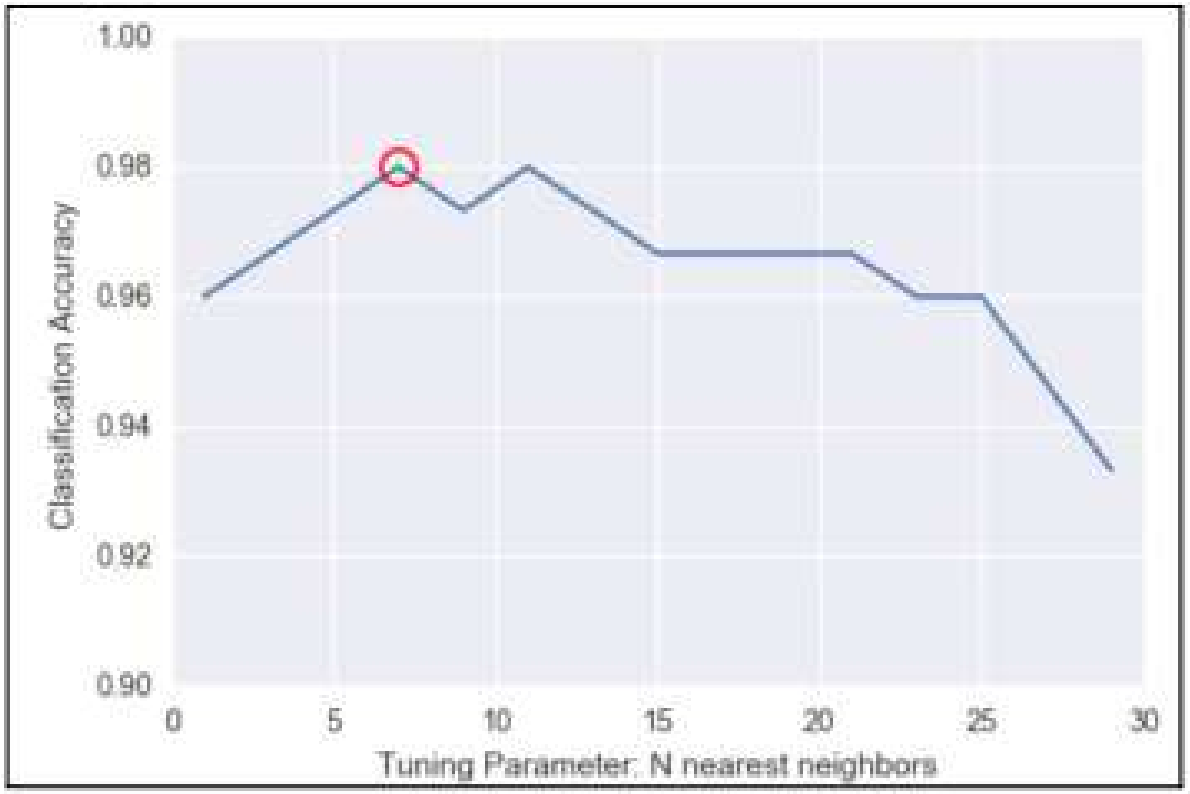
Grid searching

- sklearn also has, up its sleeve, another useful tool called grid searching.
- A grid search will by brute force try many different model parameters and give us the best one based on a metric of our choosing.
- For example, we can choose to optimize KNN for accuracy in the following manner:
- from sklearn.grid_search import GridSearchCV
- # import our grid search module
- knn = KNeighborsClassifier()
- # instantiate a blank slate KNN, no neighbors
- k_range = range(1, 30, 2)
- param_grid = dict(n_neighbors=k_range)
- # param_grid = {"n_ neighbors": [1, 3, 5, …]}

- grid = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy')
- grid.fit(X, y)

- In the grid.fit() line of code, what is happening is that, for each combination of features, in this case we have 15 different possibilities for K, we are cross-validating each one five times.
- This means that by the end of this code, we will have 15 * 5 = 75 different KNN models!
- You can see how, when applying this technique to more complex models, we could run into difficulties with time:
- # check the results of the grid search
- grid.grid_scores_
- grid_mean_scores = [result[1] for result in grid.grid_scores_]
- # this is a list of the average accuracies for each parameter
- # combination
  - plt.figure()
  - plt.ylim([0.9, 1])
  - plt.xlabel('Tuning Parameter: N nearest neighbors')
  - plt.ylabel('Classification Accuracy')
  - plt.plot(k_range, grid_mean_scores)

plt.plot(grid.best_params_['n_neighbors'], grid.best_score_, 'ro', markersize=12, markeredgewidth=1.5, markerfacecolor='None', markeredgecolor='r')



- Note that the preceding graph is basically the same as the one we achieved previously with our for loop, but much easier!
- We see that seven neighbors (circled in the preceding graph) seem to have the best accuracy. However, we can also, very easily, get our best parameters and our best model, as shown:
- grid.best_params_
- # {'n_neighbors': 7}
- grid.best_score_
- # 0.9799999999
- grid.best_estimator_
- # actually returns the unfit model with the best parameters
- # KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski', metric_params=None, n_jobs=1, n_neighbors=7, p=2, weights='uniform')

- I'll take this one step further.
- Maybe you've noted that KNN has other parameters as well, such as algorithm, p, and weights.
- A quick look at the scikit-learn documentation reveals that we have some options for each of these, which are as follows:
- • **p** is an integer and represents the type of distance we wish to use.
- By default, we use p=2, which is our standard distance formula.
- • **Weights** is, by default, *uniform*, but can also be distance, which weighs points by their distance, which means that closer neighbors have a greater impact on the prediction.
- • **Algorithm** is how the model finds the nearest neighbors. We can try *ball_ tree, kd_tree, or brute*. The default is *auto*, which tries to use the best one automatically.

- knn = KNeighborsClassifier()
  - k_range = range(1, 30)
  - algorithm_options = ['kd_tree', 'ball_tree', 'auto', 'brute']
  - p_range = range(1, 8)
  - weight_range = ['uniform', 'distance']
  - param_grid = dict(n_neighbors=k_range, weights=weight_range, algorithm=algorithm_options, p=p_range)
- # trying many more options
- grid = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy')
- grid.fit(X, y)
- The preceding code takes about a minute to run on my laptop because it is trying many, 1, 648, different combinations of parameters and cross-validating each one five times.
- All in all, to get the best answer, it is fitting 8,400 different KNN models!
- grid.best_score_
- 0.98666666
- grid.best_params_
- {'algorithm': 'kd_tree', 'n_neighbors': 6, 'p': 3, 'weights': 'uniform'}

- Grid searching is a simple (but inefficient) way of parameter tuning our models to get the best possible outcome.
- It should be noted that to get the best possible outcome, data scientists should use feature manipulation (both reduction and engineering) to obtain better results in practice as well.
- It should not merely be up to the model to achieve the best performance.

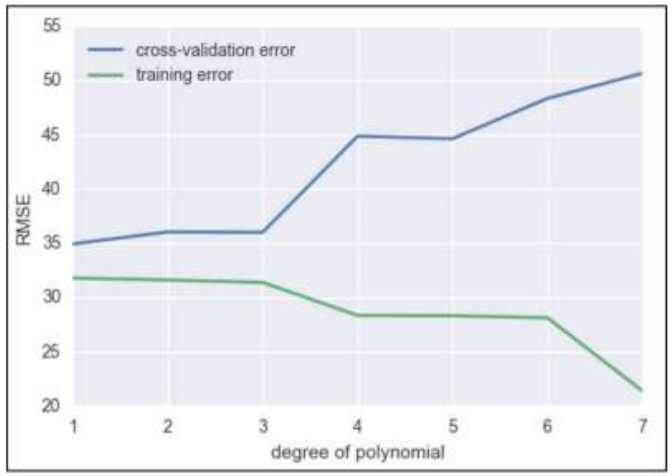
Visualizing training error versus cross-validation error

- I think it is important once again to go over and compare the cross-validation error and the training error.
- This time, let's put them both on the same graph to compare how they both change as we vary the model complexity.
- I will use the mammal dataset once more to show the cross-validation error and the training error (the error on predicting the training set).
- Recall that we are attempting to regress the body weight of a mammal to the brain weight of a mammal.

```
# This function uses a numpy polynomial fit function to
# calculate the RMSE of given X and y
def rmse(x, y, coefs):
    yfit = np.polyval(coefs, x)
    rmse = np.sqrt(np.mean((y - yfit) ** 2))
    return rmse

xtrain, xtest, ytrain, ytest = train_test_split(df['body'], df['brain'])

train_err = []
validation_err = []
degrees = range(1, 8)
for i, d in enumerate(degrees):
    p = np.polyfit(xtrain, ytrain, d)
    # built in numpy polynomial fit function
    train_err.append(rmse(xtrain, ytrain, p))
    validation_err.append(rmse(xtest, ytest, p))
  fig, ax = plt.subplots()
  # begin to make our graph
  ax.plot(degrees, validation_err, lw=2, label = 'cross-validation error')
  ax.plot(degrees, train_err, lw=2, label = 'training error')
  # Our two curves, one for training error, the other for cross validation
ax.legend(loc=0)
ax.set_xlabel('degree of polynomial')
ax.set_ylabel('RMSE')
```



- So, we see that as we increase our degree of fit, our training error goes down without a hitch, but we are now smart enough to know that as we increase the model complexity, our model is overfitting to our data and is merely regurgitating our data back to us, whereas our cross validation error line is much more honest and begins to perform poorly after about degree 2 or 3.

- To recap:
- • Underfitting occurs when the cross-validation error and the training error are both high
- • Overrfitting occurs when the cross-validation error is high, while the training error is low
- • We have a good fit when the cross-validation error is low, and only slightly higher than the training error
- Both underfitting (high bias) and overfitting (high variance) will result in poor generalization of the data.

- Here are some tips if you face high bias or variance.
- If your model tends to have a *high bias*:
- • Try adding more features to the training and test sets
- • Either add to the complexity of your model or try a more modern sophisticated model.
- If your model tends to have a *high variance*:
- • Try to include more training samples, which reduces the effect of overfitting.
- In general, the bias/variance tradeoff is the struggle to minimize bias and variance in our learning algorithms.
- Many newer learning algorithms, invented in the past few decades, were made with the intention of having the best of both worlds.

Ensembling Techniques

- Ensemble learning, or ensembling, is the process of combining multiple predictive models to produce a supermodel that is more accurate than any individual model on its own.
- • **Regression:** We will take the average of the predictions for each model
- • **Classification:** Take a vote and use the most common prediction, or take the average of the predicted probabilities
- Imagine that we are working on a binary classification problem (predicting either 0 or 1).

- # ENSEMBLING
- import numpy as np
- # set a seed for reproducibility
- np.random.seed(12345)
- # generate 1000 random numbers (between 0 and 1) for each model, representing 1000 observations
- mod1 = np.random.rand(1000)
- mod2 = np.random.rand(1000)
- mod3 = np.random.rand(1000)
- mod4 = np.random.rand(1000)
- mod5 = np.random.rand(1000)
- Now, we simulate five different learning models that each have about a 70% accuracy, as follows:
- # each model independently predicts 1 (the "correct response") if random number was at least 0.3
- preds1 = np.where(mod1 > 0.3, 1, 0)
- preds2 = np.where(mod2 > 0.3, 1, 0)
- preds3 = np.where(mod3 > 0.3, 1, 0)
- preds4 = np.where(mod4 > 0.3, 1, 0)
- preds5 = np.where(mod5 > 0.3, 1, 0)

- print preds1.mean()
- 0.699
- print preds2.mean()
- 0.698
- print preds3.mean()
- 0.71
- print preds4.mean()
- 0.699
- print preds5.mean()
- 0.685
- # Each model has an "accuracy of around 70% on its own
- # average the predictions and then round to 0 or 1
- ensemble_preds = np.round((preds1 + preds2 + preds3 + preds4 + preds5)/5.0).astype(int)
- ensemble_preds.mean()
- 0.83

- As you add more models to a voting process, the probability of errors will decrease; this is known as Condorcet's jury theorem.
- Crazy, right?
- For ensembling to work well in practice, the models must have the following characteristics:
- • **Accuracy:** Each model must at least outperform the null model
- • **Independence:** A model's prediction is not affected by another model's prediction process.
- If you have a bunch of individually *OK* models, the edge case mistakes made by one model are probably not going to be made by the other models, so the mistakes will be ignored when combining the models
- There are the following two basic methods for ensembling:
- • *Manually ensemble your individual models by writing a good deal of code*
- • *Use a model that ensembles for you*

- We're going to look at a model that ensembles for us. To do this, let's take a look at decision trees again.
- *Decision trees tend to have low bias and high variance*.
- Given any dataset, the tree can keep asking questions (making decisions) until it is able to nitpick and distinguish between every single example in the dataset.
- It could keep asking question after question until there is only a single example in each leaf (terminal) node.
- The tree is trying too hard, growing too deep, and just memorizing every single detail of our training set.
- However, if we started over, the tree could potentially ask different questions and still grow very deep.
- This means that there are many possible trees that could distinguish between all elements, which means higher variance.
- It is unable to generalize well. In order to reduce the variance of a single tree, we can place a restriction on the number of questions asked in a tree (the max_depth parameter) or we can create an ensemble version of decision trees, called Random forests.

Random forests

- The primary weakness of decision trees is that different splits in the training data can lead to very different trees.
- Bagging is a general purpose procedure to reduce the variance of a machine learning method, but is particularly useful for decision trees.
- Bagging is short for Bootstrap aggregation, which means the aggregation of Bootstrap samples.
- What is a Bootstrap sample? It is a random sample with replacement.

- # set a seed for reproducibility
- np.random.seed(1)

- # create an array of 1 through 20
- nums = np.arange(1, 21)
- print nums
- [1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]

- # sample that array 20 times with replacement
- np.random.choice(a=nums, size=20, replace=True)
- [6 12 13 9 10 12 6 16 1 17 2 13 8 14 7 19 6 19 12 11]
- # This is our bootstrapped sample notice it has repeat variables !

- So, how does bagging work for decision trees?
- 1. Grow B trees using Bootstrap samples from the training data.
- 2. Train each tree on its Bootstrap sample and make predictions.
- 3. Combine the predictions:
  - ° Average the predictions for regression trees
  - ° Take a vote for classification trees
- The following are a few things to note:
- • Each Bootstrap sample should be the same size as the original training set
- • B should be a large enough value that the error seems to have *stabilized*
- • The trees are grown intentionally deep so that they have low bias/high variance rapped sample notice it has repeat variables

- The reason we grow the trees intentionally deep is because the bagging inherently increases predictive accuracy by reducing the variance, similar to how cross validation reduces the variance associated with estimating our out of sample error.
- Random forests are a variation of bagged trees.
- However, when building each tree, each time we consider a split between the features, a random sample of *m* features is chosen as split candidates from the full set of *p* features.
- The split is only allowed to be one of those *m* features:
- • A new random sample of features is chosen for every single tree at *every single* split
- • For classification, *m* is typically chosen to be the square root of *p*
- • For regression, *m* is typically chosen to be somewhere between *p/3* and *p*
- What's the point?

- Suppose there is one very strong feature in the dataset.
- When using decision (or bagged) trees, most of the trees will use that feature as the top split, resulting in an ensemble of similar trees that are highly correlated to each other.
- If our trees are highly correlated to each other, then averaging these quantities will not significantly reduce variance (which is the entire goal of ensembling).
- Also, by randomly leaving out candidate features from each split, Random forests reduce the variance of the resulting model.
- Random forests can be used in both classification and regression problems and can be easily used in scikit-learn.
- Let's try to predict MLB salaries based on statistics about the player, as shown:

- # read in the data
- url = '../data/hitters.csv'
- hitters = pd.read_csv(url)

- # remove rows with missing values
- hitters.dropna(inplace=True)

- # encode categorical variables as integers
- hitters['League'] = pd.factorize(hitters.League)[0]
- hitters['Division'] = pd.factorize(hitters.Division)[0]
- hitters['NewLeague'] = pd.factorize(hitters.NewLeague)[0]

- # define features: exclude career statistics (which start with "C") and the response (Salary)
- feature_cols = [h for h in hitters.columns if h[0] != 'C' and h != 'Salary‘]

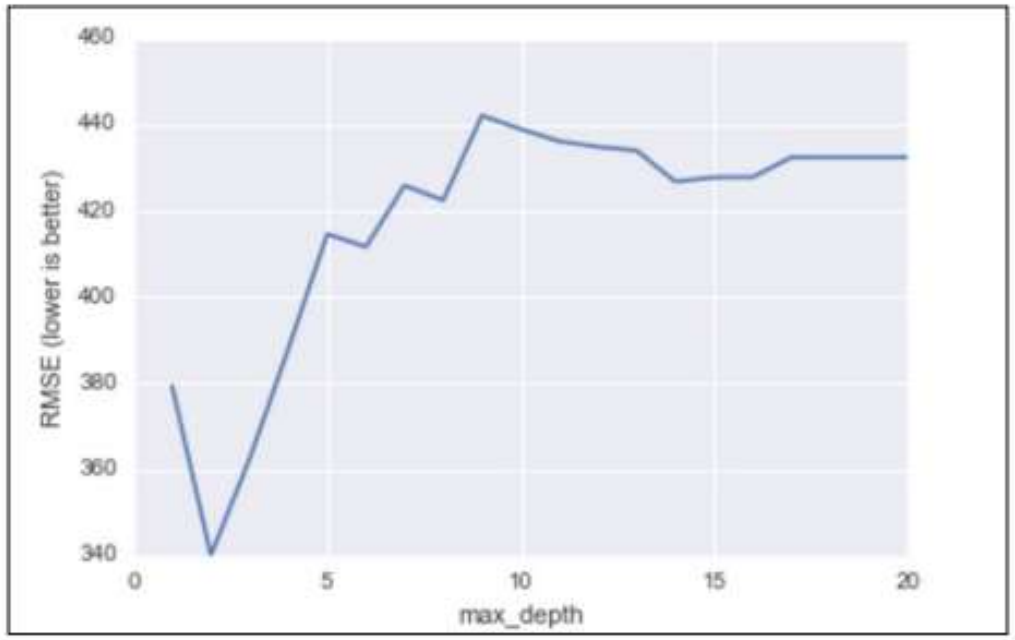
- # define X and y
- X = hitters[feature_cols]
- y = hitters.Salary
- Let's try and predict the salary first using a single decision tree, as illustrated:
- from sklearn.tree import DecisionTreeRegressor

- # list of values to try for max_depth
- max_depth_range = range(1, 21)

- # list to store the average RMSE for each value of max_depth
- RMSE_scores = []

- # use 10-fold cross-validation with each value of max_depth
- from sklearn.cross_validation import cross_val_score
- for depth in max_depth_range:
  - treereg = DecisionTreeRegressor(max_depth=depth, random_state=1)
  - MSE_scores = cross_val_score(treereg, X, y, cv=10, scoring='mean_ squared_error')
    RMSE_scores.append(np.mean(np.sqrt(-MSE_scores)))

- # plot max_depth (x-axis) versus RMSE (y-axis)
  plt.plot(max_depth_range, RMSE_scores)
- plt.xlabel('max_depth')
- plt.ylabel('RMSE (lower is better)')



RMSE for decision tree models against the max depth of the tree (complexity)

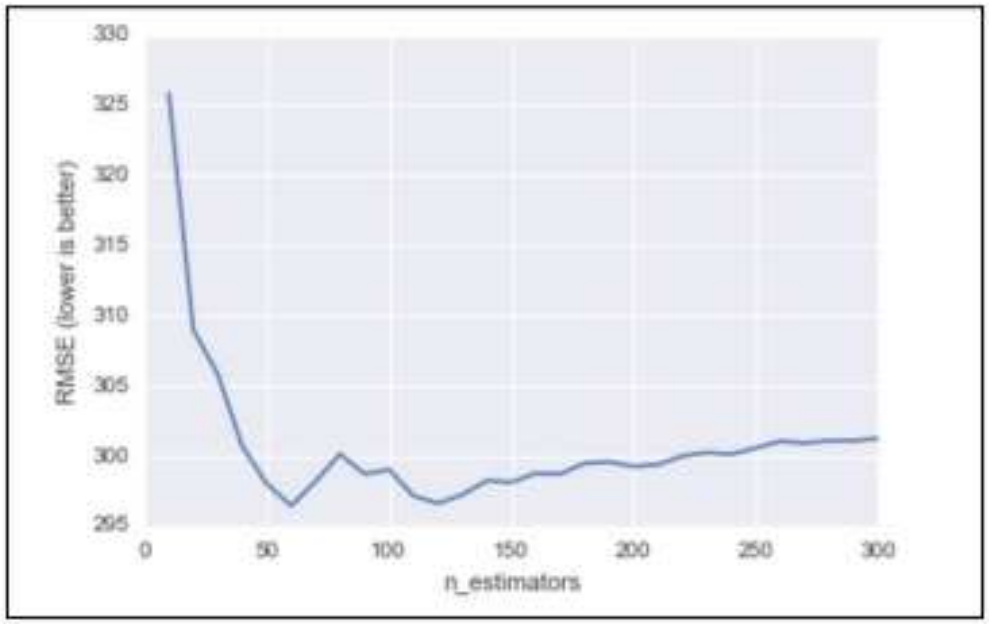
- Let's do the same thing, but this time with a Random forest:
- from sklearn.ensemble import RandomForestRegressor

- # list of values to try for n_estimators
- estimator_range = range(10, 310, 10)

- # list to store the average RMSE for each value of n_estimators
- RMSE_scores = []

- # use 5-fold cross-validation with each value of n_estimators (WARNING: SLOW!)
- for estimator in estimator_range:
  - rfreg = RandomForestRegressor(n_estimators=estimator, random_ state=1)
  - MSE_scores = cross_val_score(rfreg, X, y, cv=5, scoring='mean_ squared_error')
  - RMSE_scores.append(np.mean(np.sqrt(-MSE_scores)))

- # plot n_estimators (x-axis) versus RMSE (y-axis)
- plt.plot(estimator_range, RMSE_scores)
- plt.xlabel('n_estimators')
- plt.ylabel('RMSE (lower is better)')



RMSE for random forest models against the max depth of the tree (complexity)

- Note already the y axis, our RMSE is much lower on an average! See how we can obtain a major increase in predictive power using Random forests.
- In Random forests, we still have the concept of important features like we had in decision trees:
- # n_estimators=150 is sufficiently good
- rfreg = RandomForestRegressor(n_estimators=150, random_state=1) rfreg.fit(X, y)

- # compute feature importances
- pd.DataFrame({'feature':feature_cols, 'importance':rfreg.feature_ importances_}).sort('importance', ascending = False)

- So, it looks like the number of years the player has been in the league is still the most important feature when deciding that player's salary

	feature	importance
6	Years	0.263990
5	Walks	0.146786
1	Hits	0.139801
4	RBI	0.136265
0	AtBat	0.091551
9	PutOuts	0.060647
3	Runs	0.057460
2	HmRun	0.040183
11	Errors	0.024711
10	Assists	0.023367
8	Division	0.007628
12	NewLeague	0.004545
7	League	0.003067

Comparing Random forests with decision trees

- It is important to realize that just using random forests is not the solution to your data science problems.
- While random forests provides many advantages, many disadvantages also come with them, as listed.
- The advantages of Random forests are as follows:
- • Its performance is competitive with the best supervised learning methods
- • It provides a more reliable estimate of feature importance
- • It allows you to estimate out-of-sample errors without using train/test splits or cross-validation
- The disadvantages of Random forests are as follows:
- • It is less interpretable (cannot visualize an entire forest of decision trees)
- • It is slower to train and predict (not great for production or real-time purposes